

Střední průmyslová škola elektrotechniky a informatiky, Ostrava

11. Chatbot s napojením na umělou inteligenci

Maturitní práce

Autor práce: Jan Novotný

Vedoucí práce: RNDr. Rostislav Miarka, Ph.D.

Konzultant práce: Matyáš Boreček, Unicorn Systems a.s.

Třída: I4C

Školní rok: 2024/2025

Zadání maturitní práce

Jméno a příjmení žáka: Jan Novotný
Třída: I4C
Školní rok: 2024/2025
Obor vzdělání: 18-20-M/01 Informační technologie
Vedoucí maturitní práce: RNDr. Rostislav Miarka, Ph.D.
Konzultant maturitní práce: Matyáš Boreček, Unicorn Systems a.s.

Téma: 11. Chatbot s napojením na umělou inteligenci

Zadání maturitní práce

Cílem práce je vytvoření chatbota napojeného na umělou inteligenci, který bude sloužit k automatizovanému generování školních testů na základě předem definovaných pravidel/promptů. Chatbot bude zaměřen na předměty český jazyk a anglický jazyk. Úlohou chatbota bude sestavit dotaz na AI, která poté vygeneruje požadovaný test.

Výsledkem maturitní práce bude funkční aplikace (dle požadavků zadání), dále uživatelský manuál popisující ovládání aplikace a také písemná dokumentace popisující jednotlivé fáze návrhu a realizace aplikace.

Způsob zpracování a pokyny k obsahu a rozsahu maturitní práce:

Cíle práce:

- Vytvoření chatbot systému napojeného na AI.
- Generování testů
 - Slouží k vygenerování testů, které chatbot sestavuje a odesílá na AI
- Perzistence výsledků
 - Ukládání promptů a výsledků do SQL databáze s přehlednou mapou – Výsledky testů a s nimi spojená mapa promptů budou ukládány do databáze spolu s tagy, které popisují cestu, které popisují proces, jímž AI prošla při formulaci dotazu.
- Přílohy a rozšíření kontextu
 - Uživatelé budou mít možnost přikládat soubory, které chatbot využije k rozšíření obsahového kontextu při generování testů.
 - Každý soubor bude označen tagy a chatbot nabídne související obsah na základě volby uživatele.
- Tvorbu uživatelského manuálu
- Volitelný cíl: Implementace uživatelského rozhraní pro interakci s chatbotem.

Aplikace bude vyvinuta v jazyce Java a bude využívat relační databázi, s preferencí pro PostgreSQL, přičemž volba konkrétního typu databáze závisí na autorovi práce.

Písemná dokumentace, v rozsahu 8-12 normostran (normostrana = 1 800 znaků, tedy asi 30 řádků na stránku a 60 úhozů na řádek), bude obsahovat následující povinné části:

- Titulní strana, Anotace, Prohlášení, Obsah, Úvod, Závěr, Zdroje
- Hlavní kapitoly (mezi částmi Úvod a Závěr) s popisem jednotlivých fází návrhu a realizace aplikace

Odevzdána bude 1x elektronická verze a 1x tištěná verze práce. Pro elektronickou verzi vytvořte archiv s názvem *Třída_Příjmení_ČísloPráce.zip*, který bude obsahovat všechny soubory aplikace a veškerou dokumentaci v elektronické formě a odevzdejte ho přes aplikaci Google Classroom.

Archiv (*Třída_Příjmení_ČísloPráce.zip*) bude obsahovat:

- všechny zdrojové kódy aplikace,
- sestavenou funkční aplikaci,
- databázový export (*.sql),
- manuál pro uživatele (*.pdf) a
- elektronickou verzi písemné dokumentace (*.docx/*.odt + *.pdf).

K obhajobě maturitní práce si připravte elektronickou prezentaci o minimálním rozsahu 10 snímků.

Délka obhajoby maturitní práce před zkušební maturitní komisí je 15 minut.

Počet vyhotovení maturitní práce:	1 vyhotovení
Termín zadání maturitní práce:	15. listopad 2024
Termín odevzdání maturitní práce:	31. březen 2025

Ostrava 1. listopadu 2024

Ing. Zbyněk Pospěch
ředitel školy

Čestné prohlášení autora práce

Prohlašuji, že předložená práce je mým původním dílem, které jsem vypracoval samostatně. Veškerou literaturu a další zdroje, z nichž jsem při zpracování čerpal, v práci řádně cituji a jsou uvedeny v seznamu použité literatury a zdrojů informací.

Anotace

Tato práce představuje vývoj aplikace v jazyce *Java* s uživatelským rozhraním vytvořeným pomocí *JavaFX*, která je integrována s umělou inteligencí pro automatizovanou tvorbu testů pro studenty středních škol. Oproti existujícím řešením aplikace vyniká schopností automaticky generovat testy ve formátu **.docx*, což usnadňuje jejich archivaci a zpětné stahování díky systému databázového úložiště. Důraz je kladen také na validaci výstupů *AI*, čímž je zajištěna správnost a kvalita generovaných testů.

Poděkování

Poděkování patří především organizaci *Unicorn Systems a.s.* za možnost spolupráce a koncept této aplikace, konkrétně Matyáši Borečkovi za průběžné konzultace o vývoji aplikace, potřebné rady a pomoc při řešení komplexních problémů. Dále bych chtěl poděkovat RNDr. Rostislavu Miarkovi, Ph.D. za vedení práce, průběžné hodnocení, rady k prioritizaci jednotlivých částí aplikace a k administrační části práce.

Obsah

1	Úvod	10
2	Prohlášení o omezení odpovědnosti	12
3	Postup tvorby práce.....	13
3.1	Návrh	13
3.2	Implementace	13
3.3	Závěr práce a testování.....	13
4	Seznam použitých technologií a nástrojů.....	14
4.1	Jazyk Java a JDK 21	14
4.2	Vývojové prostředí	14
4.3	Buildovací a verzovací nástroje	14
4.4	Databázový systém.....	14
4.5	Tvorba uživatelského rozhraní	14
4.6	AI a API služby	15
4.7	Ostatní knihovny	15
5	Struktura databáze	16
5.1	E-R model.....	16
5.2	Mapování a popis tabulek.....	17
5.2.1	Tabulka users.....	17
5.2.2	Tabulka subjects.....	17
5.2.3	Tabulka users_subjects.....	18
5.2.4	Tabulka topics	18
5.2.5	Tabulka prompts	18
5.2.6	Tabulka topics_prompts.....	19
5.2.7	Tabulka tests.....	19
5.2.8	Tabulka questions.....	20
5.2.9	Tabulka questions_tests.....	20
5.2.10	Tabulka answers.....	20
5.2.11	Tabulka questions_answers.....	21
6	Souborová struktura.....	22
6.1	Kořenová složka.....	23
6.2	Složka resources	23
6.3	Složka app.....	24
6.3.1	Třída App.java	24
6.3.2	Dílčí balíček controllers	24
6.3.3	Dílčí balíček dao.....	28
6.3.4	Dílčí balíček enums	29
6.3.5	Dílčí balíček models.....	30
6.3.6	Dílčí balíček services	31
7	Pokyny pro instalaci aplikace	37
7.1	Předpoklady před instalací	37
7.2	Návod k instalaci.....	37
8	Závěr	39
8.1	Možnosti dalšího vývoje.....	39
8.2	Přínosy aplikace.....	39

Seznam obrázků

Úvodní pohled pro roli učitele	11
<i>E-R model</i> databáze	16
Pohled pro registrační formulář	26
Pohled pro vytvoření tematického celku	27
Pohled pro přehled předmětů pro roli administrátora	28
Příklad výpisů z <i>logů</i>	34
Ukázka vygenerovaného testu	36

Seznam kódů

Pohled login-view.fxml.....	23
CSS pro třídu menu-button.....	23
Metoda initialize: Dynamické nastavování barvy ikony pro odhlášení.....	24
Metoda onCreateTest: Vytvoření instance objektu Task	25
Metoda onCreateTest: Spuštění samotného Tasku a akce po jeho dokončení	25
Metoda executeUpdate z třídy EditProfileController.java.....	27
Metoda getTopics z třídy TopicManager.java	29
UserRoleEnum s konstruktorem a metodou fromString	30
Metoda sendRequestToAI: Příprava dat a navázání spojení.....	32
Metoda sendRequestToAI: Příjem a vrácení odpovědi	32
Metoda sendRequestToAI: Zpracování chyb.....	33
Statická vnořená třída OneTimeFormatter.....	34
Metoda createDocxFile z třídy FileService.java	35
Konfigurace databáze v .env	38
Konfigurace aplikace v .env	38
Konfigurace pro integraci <i>OpenAI</i> v .env	38

1 Úvod

Cílem této maturitní práce je vytvořit aplikaci, která usnadní tvorbu a správu testů pro studenty středních škol. Aplikace, implementovaná v jazyce *Java* s uživatelským rozhraním postaveným na *JavaFX*, je integrována s umělou inteligencí a umožňuje automatizované generování testů na základě přesně definovaných parametrů, jako jsou název testu, počet otázek, časový limit, úroveň obtížnosti, předmět, konkrétní témata a případně doplňující popis či příloha souboru. Výsledný test je předán přímo ve formátu **.docx*, což značně zjednodušuje samotné použití testu a případné další využití dokumentu.

Naproti běžným nástrojům volně dostupné umělé inteligence, jako jsou *ChatGPT*, *Gemini* nebo *Claude*, představuje tato aplikace výrazně propracovanější řešení specifické pro generování testů studentům střední školy. Prvním zásadním rozdílem je automatizované generování a okamžitá tvorba dokumentu ve formátu **.docx* – funkce, kterou nelze dosáhnout pomocí běžných volně přístupných *AI* systémů.

Dalším klíčovým aspektem je způsob získávání vstupních dat. *Promptování*, tedy formulace vstupních požadavků pro *AI*, se stalo dovedností, kterou ovládne jen málo uživatelů. Mnoho uživatelů při psaní *promptu* opomene specifikovat řadu důležitých parametrů, což vede k ne zcela uspokojivým výsledkům. Moje aplikace vyžaduje, aby uživatel zadal přesně definované informace, přičemž některé údaje, například fakt, že se jedná o test pro žáky střední školy, jsou do *promptu* integrovány automaticky a uživatel je nemusí zadávat opakovaně, což výrazně šetří čas. Díky tomu je umělá inteligence vedena k přesnému pochopení požadavků, což vede ke chtěnému výsledku.

Třetím významným rozdílem je implementovaná validace výstupních dat. I u rozsáhlého a přesně definovaného *promptu* není zaručeno, že *AI* na první pokus vygeneruje test ve správném formátu. Aplikace proto obsahuje validační metody, které umožňují opakované předání výsledku umělé inteligenci s konkrétními instrukcemi k úpravám, dokud výsledný dokument nevyhoví stanoveným kritériím. Je třeba zmínit, že v některých případech může dojít k tzv. *AI halucinaci*, kdy umělá inteligence začne generovat nesouvisející nebo chybné informace, nebo nedokáže naformátovat výstup správně i po několika upozorněních. V takové situaci obvykle není možné *AI* navést ke správnému výsledku během probíhajícího chatu a je třeba zahájit novou komunikaci.

Moje aplikace tento jev detekuje, informuje uživatele o vzniklé situaci a vyzve ho k opětovnému vygenerování testu. Na pozadí aplikačního řešení se tak odehrává složitý proces *promptování* a ladění, aniž by o tom uživatel věděl, což výrazně usnadňuje a zefektivňuje jeho práci.

Tímto přístupem aplikace posouvá možnosti využití umělé inteligence v oblasti vzdělávání, neboť kombinuje automatizaci, přesnost zadávání parametrů a robustní validaci výstupů tak, aby byl výstup správný a přehledný. Tento systém je možno libovolně rozšiřovat a upravovat pro konkrétní potřeby. Celá aplikace by mohla přispět ke zvýšení efektivity, tvorbě a správě testových materiálů. Níže vidíte pro příklad ukázkou úvodního pohledu pro samotné generování testů.

Generátor testů

[Generovat test](#) [Přehled testů](#) [Přidat téma](#) [Přehled témat](#) [Můj účet](#) [↗](#)

Generátor Testů

* povinné pole

Název testu *

Počet otázek *

Časový limit v minutách *

Soubor pro specifikaci kontextu

Vyberte obtížnost *

Vyberte typ otázek *

Vyberte předmět *

Vyberte předmět než vyberete témata

Další specifikace testu

Vygeneruj test

Obrázek 1: Úvodní pohled pro roli učitele

2 Prohlášení o omezení odpovědnosti

I přes pečlivou implementaci validačních metod a testování správnosti otázek a odpovědí mohou v některých případech nastat nepředvídatelné chyby související s *AI halucinacemi* či jinými odchylkami od očekávaného výstupu. Tyto chyby mohou způsobit, že test nebude vygenerován vůbec, bude vygenerován s mírnými formátovacími nedostatky, nebo může obsahovat faktické chyby či dezinformace.

Když se test nevygeneruje vůbec, jedná se pravděpodobně o výsledek *AI halucinace*, které je obtížné zabránit, neboť umožněných variant odpovědí ze strany umělé inteligence je nekonečně mnoho. Pokud dojde k drobnému formátovacímu nedostatku, může být tato chyba následně opravena laděním na základě logů, které aplikace vypisuje. V případě faktických chyb či dezinformací se aplikace rovněž snaží tyto problémy hlídat pomocí *AI validace*, avšak konečná kontrola správnosti testu je vždy na uživateli.

Používáním této aplikace tudíž uživatel bere na vědomí, že výsledný test je nutno pečlivě přezkoumat a ověřit, zda je zcela bez chyb.

3 Postup tvorby práce

3.1 Návrh

Prvním krokem bylo sepsání hrubého návrhu celkového konceptu. V tomto návrhu jsem specifikoval použitý programovací jazyk, vývojové prostředí (*IDE*), konkrétní cíle, návrh databázové a souborové struktury, koncept vzhledu uživatelského rozhraní, napojení na *AI* a očekávané výstupy aplikace. Následně jsme tento kompletní návrh projednali na jedné ze schůzek ve firmě *Unicorn systems a.s.*, kde jsme společně stanovili jak nedostatky, tak silné stránky návrhu.

Po této konzultaci jsem pokračoval v návrhu databáze, jelikož na ní celá aplikace stojí. Dále jsem se pustil do tvorby jádra aplikace – realizoval jsem kompletní souborovou strukturu, vytvořil jednotlivé modely pro každou databázovou tabulku a k nim příslušné manažery, které tyto modely spravují v databázi.

3.2 Implementace

Následovaly implementace funkcí, jako je samotné generování testů (původně pouze ve formátu **.txt*), rozsáhlá validace výstupu od *AI*, ukládání jednotlivých objektů do databáze, správa uživatelů, realizace specifických operací pro učitele i administrátora a funkce pro generování dokumentů ve formátu **.docx*.

3.3 Závěr práce a testování

Na závěr jsem sepsal dokumentaci celého projektu a uživatelský manuál aplikace. V závěrečné fázi jsem se také snažil odstranit některé nedokonalosti a průběžně testoval funkčnost celé aplikace.

Ohledně postupu práce a případných problémů jsme se scházeli každé dva až tři týdny ve firmě s Matyášem Borečkem – konzultantem mé práce. Probírali jsme aktuální stav projektu, plány do budoucna i řešení komplexních problémů. Matyáš byl pro mě velká pomoc a tímto mu ještě jednou děkuji.

4 Seznam použitých technologií a nástrojů

4.1 Jazyk Java a JDK 21

Java je objektově orientovaný programovací jazyk, který se široce využívá pro vývoj aplikací různých typů – desktopových, webových či mobilních. Právě z tohoto důvodu jsem tento jazyk zvolil pro tvorbu aplikace.

JDK (Java Development Kit) je základní balík nástrojů a knihoven, které jsou nezbytné pro vývoj a běh aplikací psaných v jazyce *Java*. V této aplikaci se využívá *JDK* verze 21, což představuje jednu z jeho nejnovějších verzí, a nabízí tedy aktuální funkce a optimalizace, které přispívají k efektivnějšímu vývoji a vyššímu výkonu aplikací.

4.2 Vývojové prostředí

IntelliJ IDEA je přední integrované vývojové prostředí (*IDE*) od společnosti *JetBrains*, které podporuje řadu programovacích jazyků, se zvláštním důrazem na *Javu*. Poskytuje například inteligentní asistenci, *refactoring* a *debugging*, což značně urychluje vývoj softwaru.

Scene Builder je vizuální nástroj pro tvorbu a úpravu *FXML* souborů, které definují grafické rozhraní v aplikacích založených na *JavaFX*. Umožňuje návrhářům a vývojářům snadno vytvářet uživatelská rozhraní pomocí drag-and-drop prvků.

4.3 Buildovací a verzovací nástroje

Závislost (dependency) označuje externí knihovnu nebo balík softwaru, který je nezbytný pro správnou funkci a kompilaci vašeho projektu.

Apache Maven je buildovací a projektový nástroj, který se používá ke správě závislostí (*dependencies*), kompilaci zdrojového kódu, testování a balení výsledných projektů. V mém projektu je využíván soubor `pom.xml` k definování závislostí a konfiguraci build procesu.

Git je distribuovaný verzovací systém, který umožňuje sledovat změny v kódu, spolupracovat v týmech a snadno spravovat různé verze vývoje softwaru. V dnešní době je široce rozšířený a je nezbytným nástrojem pro správu zdrojových kódů a historii projektu.

4.4 Databázový systém

MySQL je relační databázový systém pro ukládání a správu dat. Je známý svou spolehlivostí, robustností i vysokým výkonem. V aplikaci slouží k uložení a správě dat, jako jsou informace o testech, promptech, uživateli, předmětech i dalších entitách. Dopodrobna bude databáze popsána níže v této dokumentaci.

MySQL Workbench, který využívám pro usnadnění práce s databází, je pokročilé grafické uživatelské rozhraní, které umožňuje vizuální návrh databáze, správu a ladění *SQL* dotazů, monitorování výkonu a celkovou správu databázového systému.

XAMPP, který využívám pro práci se serverem *MySQL*, je balíček softwarových nástrojů určených pro lokální vývoj a testování webových i desktopových aplikací. *XAMPP* obsahuje *Apache*, *MySQL*, *PHP* a *Perl*, a díky svému jednoduchému nastavení umožňuje rychle nasadit a spravovat databázový server v lokálním prostředí.

4.5 Tvorba uživatelského rozhraní

JavaFX je moderní platforma pro tvorbu uživatelských rozhraní v *Javě*. Podporuje práci s grafikou, multimédií a animacemi a je často základem pro desktopové aplikace.

FXML je deklarativní jazyk založený na *XML* pro popis grafického uživatelského rozhraní v *JavaFX*. Umožňuje oddělit design *UI* od aplikační logiky, což přispívá k lepší udržitelnosti a přehlednosti kódu.

JavaFX Maven Plugin usnadňuje správu a spouštění *JavaFX* aplikací pomocí *Apache Maven*. *Plugin* integruje spouštěcí mechanismus *JavaFX* do *Maven build* procesu, což umožňuje snadné spouštění a balení aplikace.

CSS (*Cascading Style Sheets*) v kontextu *JavaFX* se od standardního *CSS* liší především tím, že je speciálně optimalizované pro stylování grafických komponent (*Node*) v prostředí *JavaFX*. Zatímco standardní *CSS* se primárně používá pro webové stránky a má předdefinovaný soubor vlastností pro *HTML* elementy, *CSS* pro *JavaFX* obsahuje rozšířenou sadu vlastností, které odpovídají specifickým atributům a vlastnostem *JavaFX* ovládacích prvků. Struktura a syntaxe je tedy mírně jiná, aby efektivně pracovala s hierarchií uzlů v *JavaFX* scéně, což umožňuje detailní a flexibilní úpravu vzhledu desktopových aplikací postavených na této platformě.

4.6 AI a API služby

OpenAI gpt-3.5-turbo je pokročilý jazykový model využívaný pro generování textu a konverzační služby. Je integrován do aplikace pro využití schopností zpracování přirozeného jazyka. Momentálně se jedná o nejlevnější volně dostupný model.

OpenAI API poskytuje přístup k modelům umělé inteligence pro integraci inteligentních funkcí do aplikací. Pomocí poskytnutého *API* klíče se aplikace připojuje k *API* na definované *URL* adrese.

4.7 Ostatní knihovny

Gson (`com.google.code.gson`) je knihovna od společnosti *Google* pro *parsování* *Java* objektů do formátu *JSON* a naopak. Umožňuje snadnou konverzi dat mezi *Java* objekty a jejich reprezentací ve formátu *JSON*, což je využíváno v komunikaci s *API* a další manipulaci s daty.

Dotenv (`io.github.cdimascio.dotenv.java`) je knihovna umožňující načítání konfiguračních proměnných z prostředí (`.env` souboru). To usnadňuje správu citlivých informací a konfigurací bez nutnosti vypisovat je natvrdo (tzv. *hardcodovat*) je přímo do kódu.

Apache POI (`org.apache.poi.xwpf.usermodel`) je knihovna, která umožňuje vytvářet a manipulovat s dokumenty ve formátech *Microsoft Office*. V projektu je využívána ke generování testů ve formátu **.docx*, přičemž umožňuje dynamické vytváření dokumentů s formátovaným textem a dalšími prvky.

Java Util Logging (`java.util.logging`) je knihovna pro *logování*, která je součástí jádra *Javy*. Umožňuje zaznamenávat, formátovat a směřovat logovací zprávy (nejen) do souborů s příponou **.log*.

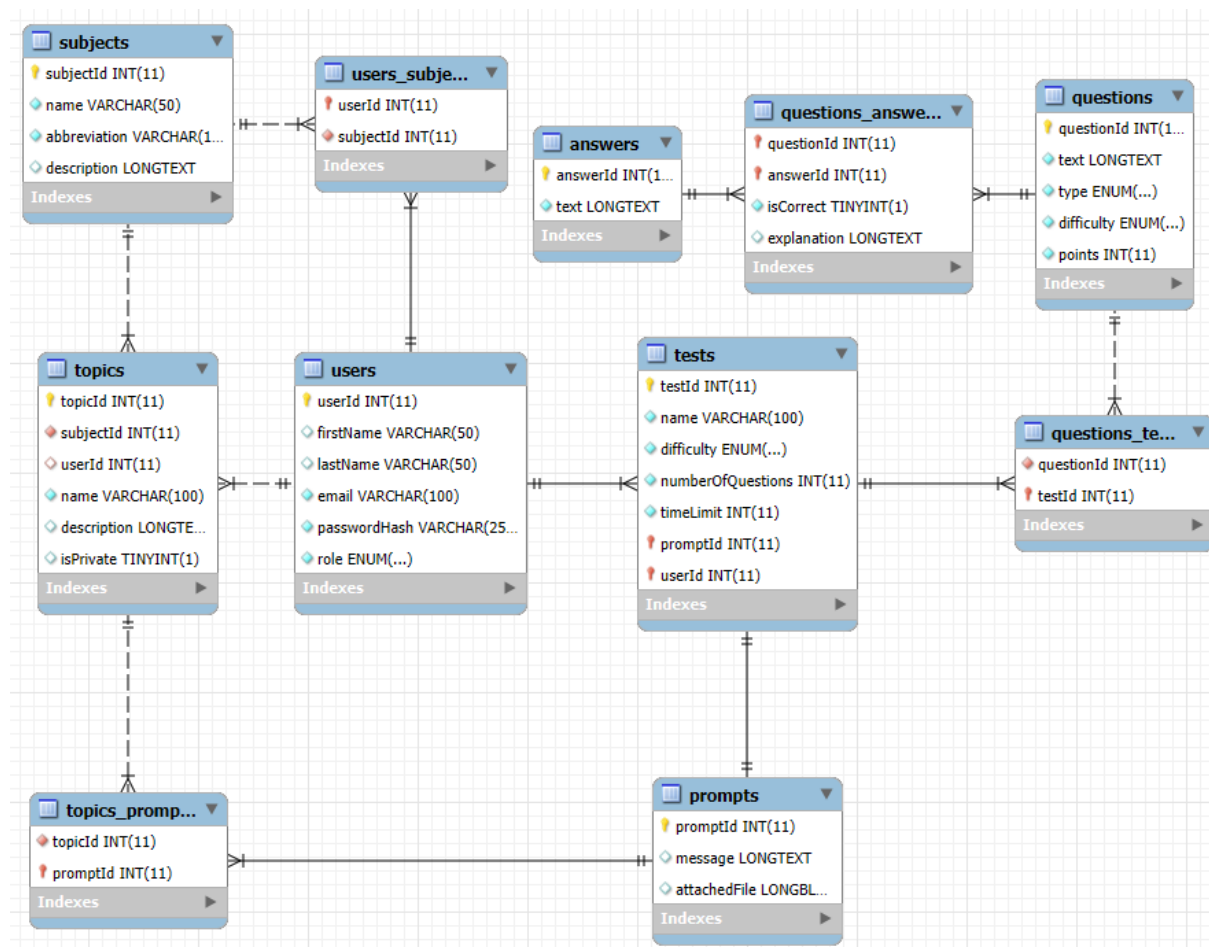
5 Struktura databáze

Než rozeberu souborovou strukturu, funkčnosti, třídy, jejich metody a konkrétní ukázky kódů aplikace, představím samotnou databázi, na které celá aplikace stojí a bez které by neměla šanci správně fungovat.

5.1 E-R model

E-R model přikládám, protože slouží jako klíčový nástroj při návrhu databáze. Umožňuje vizuálně definovat strukturu dat, což usnadňuje analýzu, optimalizaci a komunikaci mezi vývojáři. Díky *E-R modelu* lze jednoznačně identifikovat entity, jejich atributy a vztahy mezi nimi, což přispívá ke správnému návrhu a implementaci relační databáze, na které aplikace stojí.

Nutno je podotknout, že *E-R model* jsem vygeneroval pomocí funkce *Reverse engineer* v *MySQL Workbench*. Jelikož tato funkce není stoprocentně dokonalá, *E-R model* není do detailu přesný. Pro naše účely však postačí.



Obrázek 2: E-R model databáze

5.2 Mapování a popis tabulek

5.2.1 Tabulka users

Atribut	Datový typ	Povinnost	Typ klíče
userId	INT(11)	Ano	Primární
firstName	VARCHAR(50)	Ne	
lastName	VARCHAR(50)	Ne	
email	VARCHAR(100)	Ano	Unikátní
passwordHash	VARCHAR(255)	Ano	
role	ENUM('Administrátor', 'Učitel')	Ano	

userId: Unikátní identifikátor uživatele

firstName: Křestní jméno uživatele, nepovinné

lastName: Příjmení uživatele, nepovinné

email: Unikátní email uživatele

passwordHash: Heslo uživatele zašifrované pomocí algoritmu *SHA-256*

role: Role uživatele (výčtový datový typ)

5.2.2 Tabulka subjects

Atribut	Datový typ	Povinnost	Typ klíče
subjectId	INT(11)	Ano	Primární
name	VARCHAR(50)	Ano	Unikátní
abbreviation	VARCHAR(3)	Ano	Unikátní
description	LONGTEXT	Ne	

subjectId: Unikátní identifikátor předmětu

name: Unikátní název předmětu

abbreviation: Unikátní zkratka předmětu

description: Popis předmětu, nepovinný

5.2.3 Tabulka users_subjects

Atribut	Datový typ	Povinnost	Typ klíče
userId	INT(11)	Ano	Primární, Cizí (subjects)
subjectId	INT(11)	Ano	Primární, Cizí (subjects)

Tato tabulka definuje vazbu M ku N mezi tabulkami users a subjects. Zde bych rád podotknul, že uživatelé s rolí 'Administrátor' nemají přiřazené žádné předměty. Možnost pro tohoto uživatele editovat a mazat tyto předměty je vyřešena jednoduchými *SQL* dotazy přímo v aplikaci.

userId: Identifikátor uživatele (učitele), který daný předmět učí

subjectId: Identifikátor předmětu, který učí daný uživatel (učitel)

5.2.4 Tabulka topics

Atribut	Datový typ	Povinnost	Typ klíče
topicId	INT(11)	Ano	Primární
subjectId	INT(11)	Ano	Cizí (subjects)
userId	INT(11)	Ne	Cizí (users)
name	VARCHAR(100)	Ano	Unikátní
description	LONGTEXT	Ne	
isPrivate	TINYINT(1)	Ne	

topicId: Unikátní identifikátor tematického celku

subjectId: Identifikátor předmětu, ke kterému tematický celek patří

userId: Identifikátor uživatele, který tematický celek vytvořil, nepovinný atribut (ukládá se pouze tehdy, když je isPrivate nastaven na true)

name: Unikátní název tematického celku

description: Popis tematického celku, nepovinný atribut

isPrivate: Binární hodnota obsahující informaci o tom, zda se jedná o soukromý tematický celek

5.2.5 Tabulka prompts

Atribut	Datový typ	Povinnost	Typ klíče
promptId	INT(11)	Ano	Primární
message	LONGTEXT	Ne	
attachedFile	LOB	Ne	

promptId: Unikátní identifikátor *promptu*

message: Zpráva pro *AI* pro specifikaci požadavků, nepovinný atribut

attachedFile: Připojený soubor, taktéž pro specifikaci požadavků, nepovinný atribut

5.2.6 Tabulka topics_prompts

Atribut	Datový typ	Povinnost	Typ klíče
topicId	INT(11)	Ano	Primární, Cizí (topics)
promptId	INT(11)	Ano	Primární, Cizí (prompts)

Tato tabulka definuje vazbu M ku N mezi tabulkami topics a prompts. Uživatel může v *promptu* říct, že chce, aby test byl vygenerován z několika tematických celků. Stejně tak tematický celek může být zahrnut v několika promptech.

topicId: Identifikátor tematického celku, který je zahrnut v *promptu*

promptId: Identifikátor *promptu*, do kterého je tematický celek zahrnut

5.2.7 Tabulka tests

Atribut	Datový typ	Povinnost	Typ klíče
testId	INT(11)	Ano	Primární
name	VARCHAR(100)	Ano	
difficulty	ENUM('Lehká', 'Těžká', 'Střední')	Ano	
numberOfQuestions	INT(11)	Ano	
timeLimit	INT(11)	Ano	
promptId	INT(11)	Ano	Cizí (prompts)
userId	INT(11)	Ano	Cizí (users)

testId: Unikátní identifikátor testu

name: Název testu

difficulty: Obtížnost testu (výčtový datový typ)

numberOfQuestions: Počet otázek v testu

timeLimit: Časový limit v minutách na vyplnění testu

promptId: Identifikátor *promptu*, ze kterého test vznikl

userId: Identifikátor uživatele, který test vytvořil

5.2.8 Tabulka questions

Atribut	Datový typ	Povinnost	Typ klíče
questionId	INT(11)	Ano	Primární
text	LONGTEXT	Ano	Unikátní
type	ENUM('Ano / Ne', 'Výběr z odpovědí', 'Otevřená otázka')	Ano	
difficulty	ENUM('Lehká', 'Těžká', 'Střední')	Ano	
points	INT(11)	Ano	

questionId: Unikátní identifikátor otázky

text: Samotný text otázky

type: Typ otázky (výčtový datový typ)

difficulty: Obtížnost otázky (výčtový datový typ)

points: Počet bodů za tuto otázku

5.2.9 Tabulka questions_tests

Atribut	Datový typ	Povinnost	Typ klíče
questionId	INT(11)	Ano	Primární, Cizí (questions)
testId	INT(11)	Ano	Primární, Cizí (tests)

Tato tabulka definuje vazbu M ku N mezi tabulkami questions a tests. Po vygenerování testu se automaticky *parsují* a ukládají jednotlivá data do databáze. Mezi tato data patří právě otázky a jejich vazby na konkrétní test, jejich odpovědi, vysvětlení odpovědí a podobně.

questionId: Identifikátor otázky, kterou obsahuje daný test

testId: Identifikátor testu, který obsahuje danou otázku

5.2.10 Tabulka answers

Atribut	Datový typ	Povinnost	Typ klíče
answerId	INT(11)	Ano	Primární
text	LONGTEXT	Ano	Unikátní

answerId: Unikátní identifikátor odpovědi

text: Samotný text odpovědi, unikátní atribut (není důvod ukládat více totožných otázek, mohou se použít již existující)

5.2.11 Tabulka questions_answers

Atribut	Datový typ	Povinnost	Typ klíče
questionId	INT(11)	Ano	Primární, Cizí (questions)
answerId	INT(11)	Ano	Primární, Cizí (answers)
isCorrect	TINYINT(1)	Ano	
explanation	LONGTEXT	Ne	

Tato tabulka definuje vazbu M ku N mezi tabulkami questions a answers. Každá otázka může mít více odpovědí, protože aplikace umožňuje generovat testy s více možnostmi. Pouze jedna z těchto odpovědí musí být správná a tato správná odpověď bude mít přiřazeno vysvětlení.

questionId: Identifikátor otázky, ke které je přiřazena odpověď

answerId: Identifikátor odpovědi, která je přiřazena otázce

isCorrect: Binární hodnota obsahující informaci o tom, zda je daná odpověď správná pro příslušnou otázku

explanation: Vysvětlení dané odpovědi v případě, že hodnota isCorrect je true

6 Souborová struktura

```
lib/
├─ mysql-connector-j-9.1.0.jar/
logs/
├─ chatbot-2025-02-02.log
├─ chatbot-2025-03-02.log
├─ (...)
project-documentation/
├─ dokumentace.docx
├─ er-model.png
├─ uzivatelsky-manual.pdf
├─ (...)
src/
├─ main/
│   ├── java/
│   │   ├── app/
│   │   │   ├── App.java
│   │   │   ├── controllers/
│   │   │   │   ├── LoginController.java
│   │   │   │   ├── MainController.java
│   │   │   │   ├── (...)
│   │   │   ├── dao/
│   │   │   │   ├── AnswerManager.java
│   │   │   │   ├── PromptManager.java
│   │   │   │   ├── QuestionManager.java
│   │   │   │   ├── SubjectManager.java
│   │   │   │   ├── (...)
│   │   │   ├── enums/
│   │   │   │   ├── DifficultyEnum.java
│   │   │   │   ├── QuestionTypeEnum.java
│   │   │   │   ├── SubjectEnum.java
│   │   │   │   ├── TopicEnum.java
│   │   │   │   ├── UserRoleEnum.java
│   │   │   │   ├── ViewEnum.java
│   │   │   ├── models/
│   │   │   │   ├── Answer.java
│   │   │   │   ├── Prompt.java
│   │   │   │   ├── Question.java
│   │   │   │   ├── Subject.java
│   │   │   │   ├── Test.java
│   │   │   │   ├── Topic.java
│   │   │   │   ├── User.java
│   │   │   ├── services/
│   │   │   │   ├── AIService.java
│   │   │   │   ├── AITestGeneratorService.java
│   │   │   │   ├── AIValidatorService.java
│   │   │   │   ├── AlertService.java
│   │   │   │   ├── DatabaseService.java
│   │   │   │   ├── DynamicService.java
│   │   │   │   ├── FileService.java
│   │   │   │   ├── LoaderService.java
│   │   │   │   ├── LogService.java
│   │   └── resources/
│   │       ├── app/
│   │       │   ├── css/
│   │       │   │   └─ style.css
│   │       │   ├── fxml/
│   │       │   │   ├── login-view.fxml
│   │       │   │   ├── main-view.fxml
│   │       │   │   ├── (...)
│   │       └── images/
│   │           ├── logout-maroon.png
│   │           └─ logout-white.png
```

Poznámka: tato struktura je ilustrační a zahrnuje pouze hlavní složky a soubory aplikace.

6.1 Kořenová složka

V kořenové složce nalezneme složky `lib`, `logs`, `project-documentation`, `src/main/java/app` (dále jen `app`) a `src/main/resources` (dále jen `resources`) a soubory jako `.env`, `.gitignore` a `pom.xml`.

Složka `lib` obsahuje *MySQL konektor*, díky kterému je možné se připojit k databázi. Ve složce `logs` najdeme veškeré *logy*, které pomocná třída `LogService` vypisuje. Její funkci rozeberu dopodrobna níže v této dokumentaci. Složka `project-documentation` obsahuje jednotlivé informace o aplikaci, jako je tato dokumentace, uživatelský manuál, E-R model, nebo osobní poznámky. Složky `app` a `resources` jsou rozsáhlejší a také je do detailu je rozeberu níže.

Soubor `.env` obsahuje kompletní konfiguraci aplikace a je její nepostradatelnou částí. Soubor `.gitignore` specifikuje, které soubory se nemají zahrnout do verzovacího systému *Git*. Soubor `pom.xml` obsahuje informace o projektu, jako jsou jeho závislosti a *pluginy*.

6.2 Složka resources

Tato složka obsahuje dílčí složky `app/fxml` (dále jen `fxml`) a `app/css` (dále jen `css`) a složku `images`. Složka `fxml` obsahuje **.fxml* soubory, které definují jednotlivé pohledy v aplikaci. Pro demonstraci jsem vybral nejjednodušší pohled v aplikaci a to `login-view.fxml`:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.scene.control.*?>
4 <?import javafx.scene.layout.*?>
5 <?import javafx.scene.text.*?>
6
7 <StackPane xmlns:fx="http://javafx.com/fxml" fx:controller="app.controllers.LoginController" styleClass="background">
8   <VBox spacing="10" alignment="CENTER" prefWidth="600" prefHeight="750">
9     <Text text="Přihlášení" styleClass="heading"/>
10    <HyperLink text="Nemáte účet? Zaregistrujte se." onAction="#onClickOnRegisterLink" styleClass="register-link"/>
11
12    <TextField fx:id="email" promptText="Email" styleClass="input" maxWidth="350"/>
13    <PasswordField fx:id="password" promptText="Heslo" styleClass="input" maxWidth="350"/>
14
15    <Button fx:id="loginButton" text="Přihlásit" onAction="#onLogin" styleClass="submit" maxWidth="350"/>
16  </VBox>
17 </StackPane>
```

Kód 1: Pohled `login-view.fxml`

Složka `css` pak obsahuje soubor `style.css`, který je jednotný pro všechny pohledy aplikace. Jak jsem zmínil výše v kapitole [Seznam použitých technologií a nástrojů](#), *CSS pro framework JavaFX* je jiné než obyčejné *CSS* používané například pro tvorbu webových aplikací, a to jak vlastnostmi, tak i syntaxí. Zde jsem jako příklad vybral jednoduchý kód, který upravuje styl pro komponenty s *CSS* třídou `menu-button`, kterou používám pro jednotlivé tlačítka v horizontální navigaci aplikace:

```
145 .menu-button {
146   -fx-background-color: transparent;
147   -fx-text-fill: white;
148   -fx-font-size: 18px;
149   -fx-padding: 10 20;
150   -fx-cursor: hand;
151   -fx-border-width: 0 0 2;
152   -fx-border-color: transparent;
153   -fx-border-style: solid;
154   -fx-transition: all 0.2s;
155 }
156
157 .menu-button:hover {
158   -fx-text-fill: maroon;
159   -fx-border-color: maroon;
160 }
```

Kód 2: *CSS* pro třídu `menu-button`

Ve složce `images` najdeme dva obrázky (`logout-maroon.png` a `logout-white.png`), které se používají jako ikony pro odhlášení. Každý je v jiné barvě, jelikož barva se dynamicky mění při přjetí po ikoně myší v metodě `initialize`, kterou najdeme téměř v každém *controlleru* aplikace:

```
51  @FXML
52  private void initialize() {
53      logoutButton.setOnMouseEntered(event -> logoutIcon.setImage(
54          new Image(getClass().getResourceAsStream("/images/logout-maroon.png"))));
55
56      logoutButton.setOnMouseExited(event -> logoutIcon.setImage(
57          new Image(getClass().getResourceAsStream("/images/logout-white.png"))));
58  }
```

Kód 3: Metoda initialize: Dynamické nastavování barvy ikony pro odhlášení

6.3 Složka app

Jak jsem zmiňoval, složka `app` je nejrozsáhlejší složkou aplikace. Proto ji věnuji samostatnou dílčí kapitolu. V této složce nalezneme třídu `App.java` a celkem pět dílčích balíčků: `controllers`, `dao`, `enums`, `models` a `services`.

6.3.1 Třída `App.java`

Třída `App.java` je hlavním vstupním bodem aplikace, která dědí z `Application` z *frameworku JavaFX*. Inicializuje databázi pomocí `DatabaseService` a načítá úvodní uživatelské rozhraní přes `LoaderService`. V metodě `stop` odpojuje databázové spojení, čímž zajišťuje čisté ukončení aplikace. Tato třída tedy zajišťuje základní inicializační a ukončovací úkony systému.

6.3.2 Dílčí balíček `controllers`

6.3.2.1 Třída `MainController.java`

Tato třída obsahuje logiku pro kód hlavního pohledu `main-view.fxml`, na kterém má uživatel možnost generovat samotné testy. Stejně jako téměř všechny ostatní `controller` třídy, i tato obsahuje metody jako `initialize`, nebo `validateInputs`. Mezi ty zajímavější metody však patří `onCreateTest`.

Metoda `onCreateTest` generuje testy a využívá více vláknového programování pro zachování responzivity uživatelského rozhraní. Níže popisují její části více dopodrobna:

1. Pokud jsou vstupy validní, uživateli se zobrazí *loader* a vytvoří se instance objektu `Task` (úkol), který běží na pozadí. Zde se připravují objekty `Prompt` a `Test` a zpracovávají se pomocí metody `handleTestProcessing`:


```

186 @FXML
187 public void onCreateTest() {
188     if (validateInputs()) {
189         showLoader(true);
190
191         // Define a Task to run in the background
192         // Jan Novotny
193         Task<Void> task = new Task<>() {
194             // Jan Novotny
195             @Override
196             protected Void call() throws SQLException, FileNotFoundException {
197                 AITestGeneratorService ai = new AITestGeneratorService();
198                 Prompt prompt = new Prompt(message.getText(), fileAttached, chosenTopics);
199                 Test test = new Test(
200                     testName.getText(),
201                     Integer.parseInt(questionCount.getText()),
202                     Integer.parseInt(timeLimit.getText()),
203                     DifficultyEnum.fromString(difficulty.getValue()),
204                     QuestionTypeEnum.fromString(questionType.getValue()),
205                     prompt,
206                     currentUser
207                 );
208
209                 String error = handleTestProcessing(test, prompt, ai);
210
211                 if (error != null) {
212                     TestManager.deleteTestData(test.getId());
213                     AlertService.showErrorAlert(testName, error);
214                 }
215                 return null;
216             }
217         };
218     }
219 }

```

Kód 4: Metoda onCreateTest: Vytvoření instance objektu Task

- Úspěšné dokončení Tasku (úkol) skryje loader a zobrazí hlášení uživateli. V případě chyby se zobrazí chybové hlášení s možností opětovného pokusu. Úkol je spuštěn ve vlastním vlákně, tímto způsobem se zobrazí loader a zůstane zobrazený, dokud probíhají procesy zpětného promptování AI a generování a ukládání souboru:

```

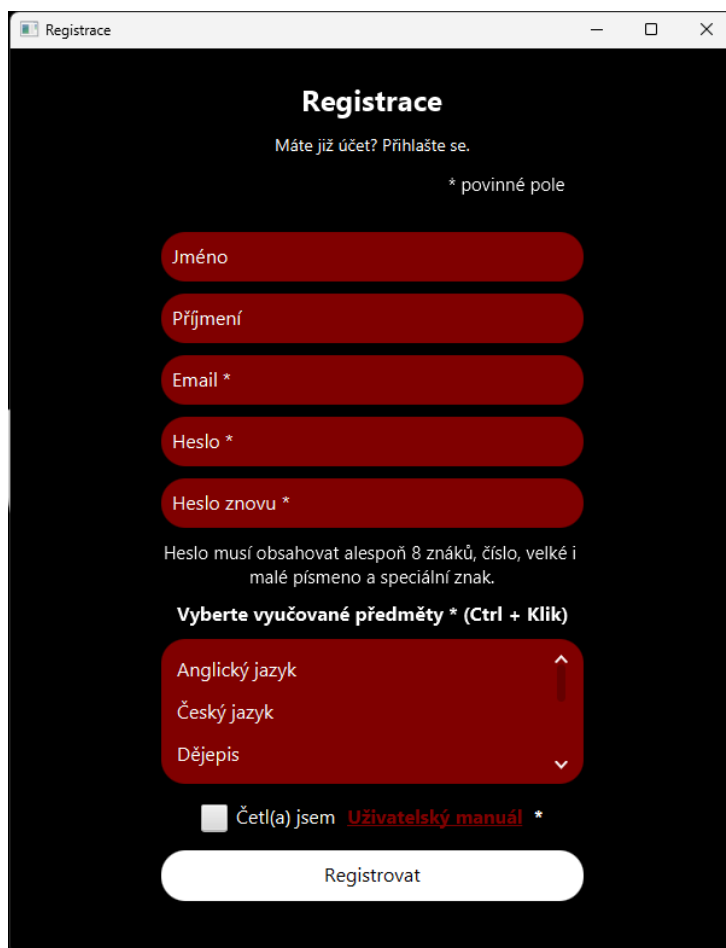
223 task.setOnSucceeded(event -> {
224     showLoader(false);
225     LogService.logInfo("Task processed successfully.");
226     AlertService.showSuccessAlert("Test byl uložen do Stažených souborů (Downloads).");
227     LoaderService.load(ViewEnum.MAIN, getClass(), testName, currentUser);
228 });
229
230 task.setOnFailed(event -> {
231     showLoader(false);
232     Throwable exception = task.getException();
233
234     if (exception != null) {
235         LogService.logError("An exception occurred: " + exception.getMessage());
236     }
237
238     AlertService.showErrorAlert(
239         "Test se nepodařilo vygenerovat z technických důvodů. Zkuste to, prosím, znovu.",
240         "Chyba!",
241         "Aplikace selhala.");
242     LoaderService.load(ViewEnum.MAIN, getClass(), testName, currentUser);
243 });
244
245 // Start the task in a new thread to keep UI responsive
246 new Thread(task).start();
247 }
248 }

```

Kód 5: Metoda onCreateTest: Spuštění samotného Tasku a akce po jeho dokončení

6.3.2.2 Třídy pro přihlášení a registraci

Jedná se o třídy `RegisterController.java` a `LoginController.java`. Tyto třídy se starají o správný běh pohledů `register-view.fxml` a `login-view.fxml`. Obsahují například metodu `validateInputs` pro zajištění správnosti uživatelem zadaných dat, nebo metody `onLogin` a `onRegister`, které řídí chod aplikace po úspěšné validaci registračních, nebo přihlašovacích údajů. `RegisterController.java` pak obsahuje například metodu `onOpenManual`, která po kliknutí na odkaz automaticky otevře Uživatelský manuál ve formátu **.pdf*.



Obrázek 3: Pohled pro registrační formulář

6.3.2.3 Třídy pro přidání instance do databáze

Jedná se o třídy `AddSubjectController.java`, `AddTopicController.java` a `AddUserController.java`, které jsou zodpovědné za funkčnost pohledů `add-subject-view.fxml`, `add-topic-view.fxml` a `add-user-view.fxml`. Tyto třídy opět obsahují metody jako `initialize`, `initializeUserData`, `validateInputs`, nebo metody pro dynamické změny pohledu.

Přidávat tematické celky může jak učitel, tak administrátor. Pohledy pro přidávání předmětů a uživatelů jsou však omezeny pouze na administrátora.

Obrázek 4: Pohled pro vytvoření tematického celku

6.3.2.4 Třídy pro editaci instance v databázi

Jedná se o třídy `EditSubjectController.java`, `EditTopicController.java` a `EditProfileController.java`, které jsou zodpovědné za funkčnost pohledů `edit-subject-view.fxml`, `edit-topic-view.fxml` a `edit-profile-view.fxml`.

Pohledy pro editaci tematického celku a uživatelského profilu jsou dostupné jak učitel, tak administrátorovi. Učitel však může upravovat pouze svůj vlastní profil. Pohledy pro úpravu předmětů jsou dostupné však pouze administrátorovi. Ten může mimo to i upravovat kterýkoliv uživatelský profil.

Mimo již zmíněné metody, které jsou stejné pro téměř všechny *controllery*, obsahují tyto třídy metody typu `executeUpdate` z `EditProfileController.java`:

```

275     private void executeUpdate(User user, String oldEmail, boolean emailChanged, boolean redirectToLogin) {
276         if (UserManager.update(user, oldEmail, emailChanged) &&
277             user.relate(subjects.getSelectionModel().getSelectedItems()) &&
278             user.unrelate(getUnselectedItems())) {
279             AlertService.showSuccessAlert("Profil byl aktualizován.");
280
281             ViewEnum viewEnum = userToEdit.getEmail().equals(currentUser.getEmail()) ?
282                 ViewEnum.EDIT_PROFILE : ViewEnum.USERS_OVERVIEW;
283
284             if (redirectToLogin) {
285                 viewEnum = ViewEnum.LOGIN;
286             }
287
288             LoaderService.load(viewEnum, getClass(), firstName, currentUser);
289         }
290     }

```

Kód 6: Metoda `executeUpdate` z třídy `EditProfileController.java`

6.3.2.5 Třídy pro prohlížení dat z databáze

Jedná se o třídy `SubjectsOverviewController.java`, `TestsOverviewController.java`, `TopicsOverviewController.java` a `UsersOverviewController.java`, které obsahují logiku pohledů `subjects-overview-view.fxml`, `tests-overview-view.fxml`, `topics-overview-view.fxml` a `users-overview-view.fxml`. Tyto pohledy slouží jako přehledy jednotlivých instancí entit z databáze.

Přehled testů a přehled tematických celků je dostupný jak učitel, tak i administrátorovi. Učitel se však zobrazí pouze omezená data. V případě tematických celků uvidí pouze ty z předmětů, které učí a zároveň nejsou vytvořeny jako soukromé jiným uživatelem. Takové tematické celky může, jak upravovat, tak i mazat. V přehledu testů opět vidí testy pouze z předmětů, které učí.

Přehled předmětů a přehled jednotlivých uživatelů je dostupný pouze administrátorovi. Ten má také právo přidávat, upravovat i mazat kterékoliv instance z tabulek předmětů, testů, tematických celků a uživatelů.

Tyto *controllery* používají statickou metodu `setCellFactory` z třídy `DynamicService.java`. Tu rozeberu více dopodrobna v kapitole [dílní balíček services](#).



Obrázek 5: Pohled pro přehled předmětů pro roli administrátora

6.3.3 Dílní balíček dao

DAO (*Data Access Object*) je návrhový vzor používaný v softwarovém inženýrství pro oddělení aplikační logiky od operací s databází. Cílem *DAO* je poskytnout abstraktní rozhraní pro práci s datovými úložišti, což usnadňuje údržbu kódu a zlepšuje jeho čitelnost. Balíček *DAO* typicky obsahuje třídy, které zprostředkovávají přístup k databázovým tabulkám a definují metody pro operace jako čtení, zápis, aktualizace a mazání dat.

Každé entitě v mé databázi pak odpovídá právě jedna třída v tomto balíčku. Například entitě `users` přísluší třída `UserManager.java`. Většina tříd v tomto balíčku obsahuje metody pro základní operace, jako jsou `save`, `update` a `delete`, které zajišťují vkládání, úpravu a mazání záznamů v databázi. Kromě toho jsou zde i metody typu `get`, které využívají *SQL* příkazu `SELECT` pro načítání dat a často se zde nacházejí v několika přetížených variantách.

Například metoda `getId(Subject subject)` se liší od metody `getId(Topic topic)`, protože každá obsahuje jako parametr jiný objekt (přetěžování metod umožňuje definovat stejné jméno metody pro různé parametry).

Podobně metody `getTopics(String fromSubject, User forUser)` a `getTopics(User forUser)` nejsou stejné, i přesto, že jsou ze stejné třídy (podmínka v dotazu `SELECT` bude u obou metod odlišná).

Použití přetížení a polymorfismu usnadňuje volání a zpřehledňuje samotný kód. Pro jednoduchou demonstraci jsem vybral již zmíněnou metodu `getTopics` z třídy `TopicManager.java`. Jak vidíme, metoda (stejně jako ostatní metody obsahující dotazy s parametry) používá objekt třídy `PreparedStatement`, čímž zabráňuje takzvaným *SQL injections*. Stejně tak používá tzv. *try-with-resources*, což zajišťuje automatické uvolnění zdrojů (v tomto případě objekty tříd `PreparedStatement` a `ResultSet`).

```

229 2 usages  ± Jan Novotný *
230 public static ArrayList<String> getTopics(String fromSubject, User forUser) throws SQLException {
231     int subjectId = SubjectManager.getId(fromSubject);
232     int userId = UserManager.getId(forUser.getEmail());
233     ArrayList<String> topics = new ArrayList<>();
234
235     String sql = "SELECT name FROM topics " +
236                 "WHERE subjectId = ? " +
237                 "AND (isPrivate = FALSE OR userId = ?)";
238
239     try (PreparedStatement pstmt = db.getConn().prepareStatement(sql)) {
240         pstmt.setInt(1, subjectId);
241         pstmt.setInt(2, userId);
242
243         try (ResultSet rs = pstmt.executeQuery()) {
244             while (rs.next()) {
245                 topics.add(rs.getString("name"));
246             }
247         } catch (Exception e) {
248             e.printStackTrace();
249         }
250     }
251     return topics;
252 }

```

Kód 7: Metoda `getTopics` z třídy `TopicManager.java`

6.3.4 Dílčí balíček enums

6.3.4.1 Výčtové datové typy pro výchozí hodnoty v databázi

Soubory `SubjectEnum.java` a `TopicEnum.java` slouží ke vložení výchozích předmětů a tematických celků do tabulek `subjects` a `topics` pomocí metod `saveDefaultSubjects` a `saveDefaultTopics`. Obsahují jednoduchý seznam položek, které se následně ukládají do databáze po spuštění aplikace.

6.3.4.2 Pomocné výčtové datové typy

Jedná se o soubory `DifficultyEnum.java`, `QuestionTypeEnum.java`, `UserRoleEnum.java` a `ViewEnum.java`. Tyto výčtové datové typy (dále jen *enumy*) v kódu značně usnadňují práci. Nemusíme totiž vypisovat řetězce natvrdo a riskovat tak překlapy, stačí použít právě tyto *enumy* a jejich předdefinované konstanty.

Každý *enum* v aplikaci reprezentuje skupinu pevných konstant, které zjednodušují práci s daty a zajišťují vyšší bezpečnost kódu. Například, *enum* `UserRoleEnum` definuje role uživatele jako `ADMIN` a `TEACHER`, kde každé z těchto konstant odpovídá lidsky čitelná hodnota, jako Administrátor nebo Učitel (což jsou konkrétní hodnoty v odpovídajícím atributu role v tabulce `users`). `ViewEnum` pak obsahuje seznam všech pohledů (*views*) v aplikaci, což usnadňuje jejich přechody a volání.

6.3.4.3 Konstruktor a atributy:

Enumy obsahují konstruktor a hodnotu v závorce za unikátní konstantou, aby mohly přiřadit čitelný řetězec k jednotlivým konstantám. Například `UserRoleEnum` má konstruktor, který přiřadí řetězec k dané roli v *enumu*.

6.3.4.4 Metoda `fromString`:

Tato metoda umožňuje převod řetězce na odpovídající hodnotu *enumu*. Používá se pro bezpečné mapování vstupních řetězců na definované konstanty, čímž se minimalizuje riziko chyb způsobených překlapy. Tento přístup zajišťuje konzistenci a bezpečnost při práci s pevně danými hodnotami v aplikaci a napomáhá zjednodušení údržby kódu.

```

1 package app.enums;
2
3 ± Jan Novotny
4 public enum UserRoleEnum {
5     ADMIN("Administrátor"), TEACHER("Učitel");
6     3 usages
7     private final String name;
8
9     // Enum constructor
10    4 usages ± Jan Novotny
11    UserRoleEnum(String name) { this.name = name; }
12
13    ± Jan Novotny
14    public String getName() { return name; }
15
16    // Returns the user role enum by name
17    6 usages ± Jan Novotny
18    public static UserRoleEnum fromString(String name) {
19        for (UserRoleEnum role : UserRoleEnum.values()) {
20            if (role.name.equalsIgnoreCase(name)) {
21                return role;
22            }
23        }
24        throw new IllegalArgumentException("No constant with name " + name + " found");
25    }
26 }

```

Kód 8: UserRoleEnum s konstruktorem a metodou fromString

6.3.5 Dílčí balíček models

Modely v mé aplikaci reprezentují jednotlivé entity v databázi. Všechny tyto třídy obsahují atributy, které vypíšu níže, dále jeden, nebo více konstruktů, metodu toString a dle potřeby getter a setter metody. Přesné významy a datové typy atributů zde již rozebírat nebudu, vychází totiž ze samotné struktury databáze, o které se více dočtete výše v kapitole [Mapování a popis tabulek](#).

6.3.5.1 Třída Answer.java

Reprezentuje odpověď na otázku v testu. Obsahuje tyto atributy:

- id
- text
- isCorrect
- explanation

6.3.5.2 Třída Prompt.java

Reprezentuje *prompt*, který uživatel v aplikaci zadal. Obsahuje tyto atributy:

- message
- attachedFile
- topics

6.3.5.3 Třída Question.java

Reprezentuje otázku v testu. Obsahuje tyto atributy:

- text
- type
- difficulty
- points
- answers
- correctAnswer
- explanation

6.3.5.4 Třída Subject.java

Reprezentuje předmět v databázi. Obsahuje tyto atributy:

- name
- abbreviation
- description

6.3.5.5 Třída *Test.java*

Reprezentuje konkrétní test vytvořený uživatelem. Obsahuje tyto atributy:

- name
- numberOfQuestions
- timeLimitInMinutes
- difficulty
- questionType
- fromPrompt
- fromUser
- id

6.3.5.6 Třída *Topic.java*

Reprezentuje tematický celek v databázi. Obsahuje tyto atributy:

- name
- description
- subject
- isPrivate
- createdBy

6.3.5.7 Třída *User.java*

Reprezentuje konkrétního uživatele. Obsahuje tyto atributy:

- id
- firstName
- lastName
- email
- passwordHash
- role

6.3.6 Dílčí balíček *services*

V tomto balíčku se nachází třídy, které obecně řečeno zjednodušují práci programátora a předcházejí opakujícímu se kódu, typicky za pomoci veřejných statických metod.

6.3.6.1 *Service třídy pro práci s AI*

Balíček *services* obsahuje mimo jiné klíčové třídy integrující umělou inteligenci od *OpenAI* do aplikace. Tyto třídy zajišťují správnou komunikaci s *AI* modelem a manipulaci s daty požadovanou pro generování testů.

Třída *AIService.java* slouží jako základní třída pro komunikaci s *OpenAI API*. Zajišťuje zasílání a přijímání dat z *AI*. Obsahuje atributy pro konfiguraci *API*, jako jsou *API* klíč a *URL*, které jsou načteny pomocí knihovny *Dotenv* z *.env* souboru. Hlavními funkcemi této třídy jsou metody *askAI*, která odesílá uživatelské dotazy do *AI* a vrací odpovědi, a *sendRequestToAI*, což je privátní metoda pro odesílání požadavků a přijímání odpovědí. Tuto metodu rozeberu po krocích, jelikož je poměrně rozsáhlá:

1. Vytvoří *JsonObject* *payload* s parametry modelu, teplotou, maximálním počtem tokenů a zprávami. Otevře *HTTP* spojení na danou *URL*, nastaví metodu na *POST*, přidá hlavičky pro *application/json* a autorizaci a umožní odesílání dat. *Payload* pak převede na byty v *UTF-8* a zapíše je do výstupního streamu.

```

71 // Private helper method to handle the request
72 2 usages  Jan Novotny
73 private String sendRequestToAI(JsonArray messagesArray) {
74     try {
75         // Prepare the payload
76         JsonObject payload = new JsonObject();
77         payload.addProperty("model", OPENAI_MODEL);
78         payload.addProperty("temperature", OPENAI_TEMPERATURE);
79         payload.addProperty("max_tokens", 3500);
80         payload.add("messages", messagesArray);
81
82         // Open the connection
83         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
84         conn.setRequestMethod("POST");
85         conn.setRequestProperty("Content-Type", "application/json");
86         conn.setRequestProperty("Authorization", "Bearer " + OPENAI_API_KEY);
87         conn.setDoOutput(true);
88
89         // Send the request
90         try (OutputStream os = conn.getOutputStream()) {
91             byte[] input = payload.toString().getBytes(StandardCharsets.UTF_8);
92             os.write(input, 0, input.length);
93         }
94     }
95 }

```

Kód 9: Metoda `sendRequestToAI`: Příprava dat a navázání spojení

2. Zkontroluje statusový kód odpovědi. Pokud je kód 200 (úspěch), načte odpověď pomocí `BufferedReader`, sestaví ji do `StringBuilder` a pak ji zpracuje jako `JsonObject`. Z odpovědi extrahuje obsah zprávy a přidá tuto odpověď do historie konverzace.

```

94 // Read the response
95 int statusCode = conn.getResponseCode();
96 if (statusCode == 200) {
97     // Success response
98     try (BufferedReader br = new BufferedReader(
99         new InputStreamReader(conn.getInputStream(), StandardCharsets.UTF_8))) {
100
101         StringBuilder responseBuilder = new StringBuilder();
102         String responseLine;
103
104         while ((responseLine = br.readLine()) != null) {
105             responseBuilder.append(responseLine.trim());
106         }
107
108         // Parse the response JSON
109         JsonObject responseJson = JsonParser.parseString(responseBuilder.toString()).getAsJsonObject();
110         String assistantResponse = responseJson
111             .getAsJsonArray("choices")
112             .get(0)
113             .getAsJsonObject()
114             .getAsJsonObject("message")
115             .get("content")
116             .getAsString();
117
118         // Add the assistant's message to the conversation history
119         JsonObject assistantMessage = new JsonObject();
120         assistantMessage.addProperty("role", "assistant");
121         assistantMessage.addProperty("content", assistantResponse);
122         conversationHistory.add(assistantMessage);
123
124         // Return the assistant's response
125         return assistantResponse;
126     }
127 }

```

Kód 10: Metoda `sendRequestToAI`: Přejím a vrácení odpovědi

3. Pokud statusový kód není 200, zpracuje chybovou odpověď, pokusí se načíst podrobnosti chyby a zapíše je do *logů*. V případě výjimky při zpracování požadavku nebo odpovědi je chyba *zalogována*. Odpověď asistenta je vrácena jako řetězec. Pokud dojde k chybě, metoda vrátí `null`.


```

127     } else {
128         // Error response
129         LogService.logError("HTTP " + statusCode);
130         try (BufferedReader br = new BufferedReader(
131             new InputStreamReader(conn.getErrorStream(), StandardCharsets.UTF_8))) {
132             StringBuilder errorBuilder = new StringBuilder();
133             String errorLine;
134             while ((errorLine = br.readLine()) != null) {
135                 errorBuilder.append(errorLine.trim());
136             }
137             LogService.logError("Details: " + errorBuilder);
138         }
139     }
140 } catch (Exception e) {
141     LogService.logError("Failed to send prompt to the OpenAI API");
142     e.printStackTrace();
143 }
144 return null;
145 }

```

Kód 11: Metoda `sendRequestToAI`: Zpracování chyb

Třída `AITestGeneratorService.java` je specializovaná třída pro generování testů pomocí umělé inteligence. Dědí od `AIService` a rozšiřuje jeho funkcionalitu. Třída zajišťuje přípravu a strukturování dat pro AI podle specifikací testu. Obsahuje metodu `getContextFor(Test test)`, která vytváří kontextový rámec pro AI generátor testů, přičemž zajišťuje dodržení formátování a parametrů, jako jsou typ otázek a obtížnost.

Třída `AIV validatorService.java` provádí validaci a kontrolu kvality výstupů od AI. Zahrnuje metodu `validateOutput`, která zkontroluje odpověď AI, identifikuje a případně opravuje chyby. Třída využívá metodu `checkFormatIssues` k detekci formátových problémů a `isFactuallyCorrect` k ověření faktické správnosti. Umožňuje efektivně spravovat a reagovat na možné nepřesnosti ve vygenerovaných testech.

6.3.6.2 Pomocné service třídy

Mezi pomocné třídy by se prakticky daly zařadit všechny z tohoto balíčku. Já sem zde zařadil ale pouze ty, které se používají prakticky v každém *controlleru* a opravdu značně usnadňují práci při psaní programu.

Třída `AlertService.java` se zaměřuje na zobrazování různých upozornění uživatelům. Obsahuje metody jako `showErrorAlert`, `showSuccessAlert`, a `showConfirmationAlert`, které vytvářejí dialogová okna pro chyby, úspěšně provedené akce a uživatelské potvrzení. Tyto metody využívají *JavaFX* k zobrazování snadno přístupných dialogů, což zlepšuje uživatelskou zkušenost a poskytuje okamžitou zpětnou vazbu.

`DynamicService.java` je zodpovědná za dynamické přizpůsobení uživatelského rozhraní. Obsahuje metody jako `setAdminNavigation` a `setCellFactory`. Tyto metody konfigurují navigaci a nastavují grafiku buněk v seznamech s užitečnými kontextovými nabídkami, což umožňuje administrátorům efektivně ovládat zobrazení aplikace podle aktuálního kontextu a požadovaných interakcí.

`LoaderService.java` usnadňuje přechody mezi různými pohledy aplikace. Metoda `load` se vyskytuje v několika přetížených variantách a umožňuje načítání různých scénářů v aplikaci. Třída zajišťuje, že každé zobrazení je správně inicializováno s potřebnými daty uživatele a specifickými *kontrolery*, což umožňuje plynulý přechod mezi různými částmi aplikace.

`LogService.java` se stará o zaznamenávání událostí a zpráv v rámci aplikace. Obsahuje metody `logInfo`, `logDebug`, `logWarning`, a `logError` pro různé úrovně logování. Třída zajišťuje, že logy jsou uchovávány v souborech a závažné chyby jsou zaznamenávány i do konzole, což je klíčové pro efektivní sledování a diagnostiku problémů během běhu aplikace. Automaticky spravuje adresář pro logy a stará se o údržbu odstraněním starých záznamů. Třída využívá `FileHandler` pro ukládání záznamů do souboru a `ConsoleHandler` pro vypisování závažných chyb i na standardní konzolový výstup.

Statická vnořená třída `OneLineFormatter` zajišťuje, že logy jsou formátovány do jedné řádky. `OneLineFormatter` poskytuje přehledné a konzistentní výstupy logů, což usnadňuje čtení a analýzu záznamů.

```

13      // Custom formatter class to ensure logs are on a single line
14      2 usages  ± Jan Novotny *
15      private static class OneLineFormatter extends Formatter {
16          1 usage
17          private static final String PATTERN = "yyyy-MM-dd HH:mm:ss";
18
19          ± Jan Novotny *
20          @Override
21          public String format(LogRecord record) {
22              SimpleDateFormat simpleDateFormat = new SimpleDateFormat(PATTERN);
23              String timestamp = simpleDateFormat.format(new Date(record.getMillis()));
24              return String.format("[%s] %s: %s\n", timestamp,
25                                  record.getLevel().equals(Level.FINE) ? "DEBUG" : record.getLevel(), record.getMessage());
26          }
27      }

```

Kód 12: Statická vnořená třída `OneTimeFormatter`

Výpisy v logu by pak mohly vypadat například takto:

```

2149 [2025-03-04 17:31:22] INFO: Looking for format issues...
2150 [2025-03-04 17:31:22] INFO: Fact-checking the test...
2151 [2025-03-04 17:31:23] INFO: Test fact-checked successfully with no errors.
2152 [2025-03-04 17:31:23] DEBUG: Number of lines: 98
2153 [2025-03-04 17:31:23] DEBUG: Line 0: Název testu: Opakování druhák
2154 [2025-03-04 17:31:23] DEBUG: Line 1: Předmět: Programování
2155 [2025-03-04 17:31:23] DEBUG: Line 2: Témata: Pointery, Dynamické pole, Projekty, Řetězce
2156 [2025-03-04 17:31:23] DEBUG: Line 3: Obtížnost: Těžká
2157 [2025-03-04 17:31:23] DEBUG: Line 4: Časový limit: 20 minut
2158 [2025-03-04 17:31:23] DEBUG: Line 5:
2159 [2025-03-04 17:31:23] DEBUG: Line 6: 1. Co je to pointer v programování?
2160 [2025-03-04 17:31:23] DEBUG: Line 7: Body: 2
2161 [2025-03-04 17:31:23] DEBUG: Line 8: a) Proměnná obsahující adresu paměti
2162 [2025-03-04 17:31:23] DEBUG: Line 9: b) Proměnná obsahující hodnotu
2163 [2025-03-04 17:31:23] DEBUG: Line 10: c) Proměnná obsahující text

```

Obrázek 6: Příklad výpisů z logů

6.3.6.3 Ostatní service třídy

Třída `DatabaseService.java` zajišťuje správu připojení k databázi. Inicializuje spojení pomocí `DriverManager` s použitím konfiguračních údajů načtených z `.env` souboru přes knihovnu `Dotenv`. Obsahuje jednotlivé metody pro vytvoření všech databázových tabulek, jako jsou `createTableSubjects`, nebo `createTableTopics`. Používá také různé kontrolní přetížené statické metody `instanceInDatabase`, které zjišťují, jestli je nějaká konkrétní instance v dané tabulce. Typicky se používá pro unikátní atributy tabulek a jsou využívány i mimo tuto třídu.

`FileService.java` se specializuje na čtení a zápis souborů. Jeho metoda `readFileContent` umožňuje načíst obsah textových souborů do `StringBuilderu`. Metody `writeTestToFile` společně s `createDocxFile` jsou zodpovědné za vytváření dokumentů ve formátu `*.docx`, přičemž využívá knihovnu `Apache POI`. Třída konstruuje dokumenty se specifickým formátováním (např. tituly, názvy, obtížnost) a ukládá je do složky `Downloads`.

Níže vidíme pouze část z metody `createDocxFile`, která je jinak poměrně rozsáhlá. *Parsuje* jednoduchý textový obsah, který převádí do více srozumitelné verze pro uživatele, která zahrnuje více úrovní nadpisu, úpravu formátování textu a podobně.

```

1 usage  Jan Novotny
54  private static XWPFDocument createDocxFile(String content, Test test) throws SQLException {
55      XWPFDocument document = new XWPFDocument();
56
57      XWPFPParagraph mainTitle = document.createParagraph();
58      mainTitle.setAlignment(ParagraphAlignment.CENTER);
59
60      XWPFRun mainTitleRun = mainTitle.createRun();
61      mainTitleRun.setText(test.getName());
62      mainTitleRun.setBold(true);
63      mainTitleRun.setFontSize(20);
64      mainTitleRun.setColor("0F4761");
65      mainTitleRun.setFontFamily("Aptos Display");
66
67      XWPFPParagraph subjectTitle = document.createParagraph();
68      subjectTitle.setAlignment(ParagraphAlignment.CENTER);
69
70      String subject = SubjectManager.getSubject(test.getPrompt().getTopics().get(0)).getName();
71
72      XWPFRun subjectTitleRun = subjectTitle.createRun();
73      subjectTitleRun.setText(subject);
74      subjectTitleRun.setFontSize(16);
75      subjectTitleRun.setColor("0F4761");
76      subjectTitleRun.setFontFamily("Aptos Display");
77
78      XWPFPParagraph topicsTitle = document.createParagraph();
79      topicsTitle.setAlignment(ParagraphAlignment.CENTER);

```

Kód 13: Metoda createDocxFile z třídy FileService.java

Vytvořený soubor by pak mohl vypadat nějak takto:

123456789101112131415

Opakování druhák

Programování

Pointery, Dynamické pole, Projekty, Řetězce

Obtížnost: Těžká

Časový limit: 20 minut

Maximální počet bodů: 20

1. Co je to pointer v programování?

Body: 2

a) Proměnná obsahující adresu paměti

b) Proměnná obsahující hodnotu

c) Proměnná obsahující text

2. Jaký je správný způsob alokace paměti pro dynamické pole v jazyce C?

Body: 3

a) Použití funkce malloc()

b) Statická alokace paměti

c) Použití funkce free()

3. K čemu se používají projekty v programování?

Body: 1

a) K organizaci a správě zdrojových souborů

b) K vykreslování grafiky

c) K ukládání dat do souborů

Obrázek 7: Ukázka vygenerovaného testu

7 Pokyny pro instalaci aplikace

Kvůli časovým a technickým komplikacím není aplikace sestavena do jednoduše spustitelného *.exe souboru. Konkrétní problém je bohužel s tím, že společnost *OpenAI* nedávno aktualizovala stanovení cen pro používání svých *AI* modelů s použitím *API* klíče. Podmínky a ceny *AI* modelů se u společnosti jako je *OpenAI* bohužel mění velice často, proto jsem se rozhodl zvolit nejlevnější model od *OpenAI* (*gpt-3.5-turbo*) a používat ho s minimálním peněžním vkladem.

Například, vygenerovaný test s 12 otázkami do předmětu programování celkem vypotřeboval asi 7000 tokenů, což mě stálo méně, než \$0,01. Pro detailní informace o cenách doporučuji navštívit oficiální stránky [OpenAI](#).

Následující dílčí kapitoly slouží jako detailní návod pro instalaci aplikace včetně vytvoření samotného *API* klíče a vložení minimálního peněžního vkladu.

7.1 Předpoklady před instalací

Před instalací aplikace se, prosím, ujistěte, že máte stáhnuté, nainstalované a funkční tyto systémy:

1. Verzovací systém *Git*
2. Libovolné prostředí pro vývoj aplikací ve *frameworku JavaFX* s preferencí pro *IntelliJ IDEA* od *JetBrains* (dále jen vývojové prostředí)
3. Libovolný software pro správu relačních databází *MySQL* s preferencí pro *MySQL Workbench* (dále jen databázové prostředí)
4. Libovolný software pro běh *MySQL* serveru s preferencí pro *XAMPP*

7.2 Návod k instalaci

Pokud nechcete vytvářet *API* klíč a zbytečně platit testování aplikace, můžete kompletně přeskočit body 1. až 5. a 13. až 16. Aplikace Vám bude fungovat i bez konfigurace pro integraci s *OpenAI*. Budete moci upravovat objekty v databázi dle Vaší uživatelské role. Aplikace Vám však bohužel neumožní generovat testy.

1. Otevřete oficiální stránky [OpenAI](#) pro tvorbu *API* klíčů. Pokud zde nemáte účet, založte si ho.
2. Klikněte na tlačítko „Create new secret key“
3. Volitelně si klíč pojmenujte a klikněte na „Create secret key“
4. Následně klikněte na „Copy“ a klíč si vložte například do Poznámkového bloku. Jakmile potvrdíte tlačítkem „Done“, ke klíči se již zpětně nedostanete.
5. V levé navigaci přejděte na sekci „Billing“. Klikněte na „Add payment details“, kde vyplníte údaje z Vaší platební karty. Poté zvolte „Add to credit balance“ a zadejte výši vkladu. Minimální vklad je v době psaní dokumentace \$5. Zde doporučuji nezaškrtnout *checkbox* pro automatické strhávání peněz z platební karty.
6. Zadejte následující příkaz do příkazového řádku na vrstvě složky, do které chcete projekt naklonovat:

```
git clone https://github.com/yenda420/chatbot.git
```
7. Projekt otevřete ve vývojovém prostředí.
8. V projektové struktuře přidejte závislost vybráním *JAR* souboru pojmenovaného `mysql-connector-j-9.1.0.jar` ze složky `lib`.
9. Ujistěte se, že Vám běží server *MySQL*.
10. V kořenové složce najdete soubor `.env-template`. Tam najdete jednoduchý popis jednotlivých proměnných, které budete muset nastavit. Soubor přejmenujte na `.env`.
11. V konfiguraci databáze nastavte správně proměnné `DB_HOST`, `DB_URL`, `DB_USER`, `DB_PASSWORD` a `DB_NAME`. Hodnoty těchto proměnných musí odpovídat Vašemu zvolenému databázovému prostředí. Zde je ukázka správného nastavení databáze:

```

1 # Database configuration
2 DB_HOST=localhost
3 DB_URL=jdbc:mysql://127.0.0.1:3306/
4 DB_USER=root
5 DB_PASSWORD=
6 DB_NAME=chatbot

```

Kód 14: Konfigurace databáze v .env

12. Dále v konfiguraci aplikace správně nastavte proměnné ADMIN_EMAIL a ADMIN_PASSWORD. Tyto proměnné definují přihlašovací údaje pro výchozího uživatele s rolí administrátor.
13. Maximální povolený počet otázek (MAX_QUESTIONS) doporučuji nechat nastavený na 25, či níže pro vyhnutí se *AI halucinacím*. Zde je příklad správně nastavené konfigurace aplikace:

```

8 # Application configuration
9 ADMIN_EMAIL=admin@chatbot.cz
10 ADMIN_PASSWORD=**123SecretPassword321**
11 MAX_QUESTIONS=25

```

Kód 15: Konfigurace aplikace v .env

14. Následně už jen stačí nastavit správně proměnné OPENAI_API_MODEL, kde zvolíte svůj preferovaný *AI* model a OPENAI_API_KEY, kde vložíte svůj *API klíč*. Zde je příklad správně nastavené konfigurace informací pro integraci s *OpenAI*:

```

13 # OpenAI configuration
14 OPENAI_MODEL=gpt-3.5-turbo
15 OPENAI_API_KEY=sk-proj-HfrZMOPHpFbVLDxcy-zacv-2roJv-617jUz
16
17 OPENAI_API_URL=https://api.openai.com/v1/chat/completions
18 OPENAI_TEMPERATURE=0.7

```

Kód 16: Konfigurace pro integraci OpenAI v .env

15. Proměnnou OPENAI_API_URL měnit nemusíte, pokud nepoužíváte specifické *API*.
16. Proměnnou OPENAI_TEMPERATURE taktéž nedoporučuji měnit, jelikož její hodnota ovlivňuje risk, který *AI* model podstupuje při generování výstupu.
17. Nyní ve vývojovém prostředí můžete spustit aplikaci.

8 Závěr

Původní záměr vytvořit aplikaci bez uživatelského rozhraní se rychle změnil, když byly definovány konkrétní požadavky a funkcionality aplikace. Tvorba a stylování uživatelského rozhraní ve *frameworku JavaFX* se tedy staly nezbytnými, což pro mě znamenalo naučit se tyto dovednosti od základů. To se významně promítlo do jednoduchosti vzhledu aplikace. I přes tyto výzvy byl odveden velký kus práce a cíl byl splněn. Přesto mě mrzí, že z časových důvodů nebylo možné implementovat řadu funkcí, které by mohly zlepšit uživatelskou zkušenost.

8.1 Možnosti dalšího vývoje

Vizuální stránka uživatelského rozhraní může být méně atraktivní a aplikace může být pro některé uživatele méně intuitivní. Chybí abstraktní třída pro volání *SQL* dotazů, soubory nejsou optimálně strukturovány a volání metod není vždy konzistentní.

Další velkou komplikací byla změna stanovení cen společností *OpenAI*. Konkrétně model *gpt-3.5-turbo*, který jsem do nedávna používal v aplikaci zadarmo, již není dostupný bez peněžního vkladu a minimálního nacenění. Tento *AI* model je sice stále cenově velmi dostupný, ale aplikace je již v tuto chvíli limitovaná. V tomto ohledu lze aplikaci rozšířit, například o načítání *API* klíče z proměnných prostředí (tzv. environmentálních proměnných).

8.2 Přínosy aplikace

Přes všechny nedostatky aplikace plní svůj účel při generování, archivaci a opětovném stahování testů. Také usnadňuje správu předmětů, tematických celků a uživatelů, což ji činí užitečným nástrojem pro učitele, kterým může významně ulehčit práci.

Během práce na této aplikaci jsem se naučil mnoho nového v jazyce *Java* a *frameworku JavaFX*, včetně používání knihoven a stanovování vlastních konvencí. Získal jsem zkušenosti s integrací s *AI*, psaním přehledného kódu, *debuggingem*, dodržováním požadavků a termínů. Jsem tedy velmi vděčný za příležitost realizovat tento projekt.

Seznam použitých pojmů a zkratek

AI (neboli *artificial intelligence* – česky *umělá inteligence*) je obor informatiky, který se zabývá tvorbou systémů schopných vykazovat rysy lidské inteligence – například učení, rozhodování a řešení problémů – prostřednictvím algoritmů a strojového učení.

AI halucinace označuje fenomén, kdy umělá inteligence generuje nesprávné, smyšlené nebo nesouvisějící informace bez opory v ověřitelných datech, což může vést k nepřesným či zavádějícím výsledkům.

Prompt je vstupní dotaz nebo instrukce, kterou uživatel poskytuje systému umělé inteligence, aby vyvolal konkrétní odpověď či akci.

Promptování je proces formulování a zadávání promptů.

XML (*eXtensible Markup Language*) je značkovací jazyk určený pro strukturované ukládání a přenos dat. Umožňuje definovat vlastní tagy a hierarchickou strukturu, což usnadňuje formátování, sdílení a interpretaci dat mezi různými systémy.

JSON (*JavaScript Object Notation*) je textový datový formát určený pro přenos a ukládání dat. Umožňuje organizaci dat do strukturovaných formátů, jako jsou objekty s key-value (klíč-hodnota) páry a pole, což usnadňuje jejich čtení jak pro lidi, tak pro počítače.

Parsování je proces analýzy vstupních dat (např. *XML* nebo *JSON* souboru) za účelem jejich rozdělení a převedení do strukturované formy, kterou může program snadno zpracovat. Tento proces umožňuje extrakci a organizaci informací z textového nebo jiného formátu.

E-R model (*Entity-Relationship model*) je grafická reprezentace databázového návrhu, která zobrazuje entity (objekty nebo koncepty) společně s jejich atributy a vztahy, které mezi sebou existují. Tento model pomáhá definovat strukturu databáze již v počáteční fázi vývoje a zajišťuje, že veškerá potřebná data a jejich propojení jsou správně navržena a dokumentována.

UI (neboli *user interface* – česky *uživatelské rozhraní*) je souhrn způsobů, jakými lidé (uživatelé) ovlivňují chování strojů, zařízení, počítačových programů či komplexních systémů.

SHA-256 (*Secure Hash Algorithm 256-bit*). Je algoritmus, který se používá ke kryptografickému zabezpečení. Kryptografické *hashovací* algoritmy vytvářejí nerozluštitelné a jedinečné hodnoty, tzv. *hashe*.

SQL (*Structured Query Language*) je standardizovaný jazyk používaný pro manipulaci a správu dat uložených v relačních databázových systémech. Umožňuje provádět operace jako vkládání, aktualizace, mazání a dotazování na data.

SQL injections je zranitelnost v aplikacích, která umožňuje útočnickovi vkládat škodlivé *SQL* příkazy do vstupních polí aplikace. Cílem je manipulovat s databází a získat neoprávněný přístup k datům.

Polymorfismus je vlastnost programovacího jazyka, speciálně v objektově orientovaném programování, která umožňuje objektům volání jedné metody se stejným jménem, ale s jinou implementací.

MVC (*Model-View-Controller*) je návrhový vzor pro strukturování aplikací, který odděluje model (data a logika), pohled (uživatelské rozhraní) a *controller* (zpracování vstupů). Pomáhá vytvářet aplikace, které jsou modulární a snadno upravitelné.

API (*Application Programming Interface*) je sada definovaných pravidel a protokolů, které umožňují různým softwarovým aplikacím komunikovat mezi sebou. *API* slouží jako rozhraní pro interakci mezi *softwarem* a nabízí předem definované funkce, které mohou vývojáři využít pro integraci a manipulaci s daty nebo službami bez nutnosti znát detaily implementace.

Framework je struktura skládající se z předem připravených knihoven, nástrojů a pravidel, které usnadňují vývoj softwarových aplikací. Poskytuje definovanou strukturu a sadu funkcionalit, které pomáhají standardizovat a urychlit proces vývoje, čímž umožňuje vývojářům se více zaměřit na specifickou logiku aplikace než na řešení opakujících se problémů.

Seznam použité literatury a zdrojů

- [1] *Umělá inteligence*. Online. Wikipedie. 2025. Dostupné z: https://cs.wikipedia.org/wiki/Um%C4%9B%C3%A1_inteligence. [cit. 2025-02-25].
- [2] MIT. *When AI Hallucinates*. Online. MIT Technology Review. 2024. Dostupné z: <https://www.technologyreview.com/search/?s=When%20AI%20Hallucinates>. [cit. 2025-02-25].
- [3] *AI Hallucinations*. Online. Towards Data Science. 2024. Dostupné z: <https://towardsdatascience.com/?s=AI+Hallucinations>. [cit. 2025-02-25].
- [4] MIT. *Prompt engeneering*. Online. MIT Technology Review. 2024. Dostupné z: <https://www.technologyreview.com/search/?s=Prompt%20engeneering>. [cit. 2025-02-25].
- [5] ORACLE CORPORATION. *Oracle Java*. Online. Oracle. 2025. Dostupné z: <https://www.oracle.com/java/technologies/downloads/?er=221886>. [cit. 2025-02-25].
- [6] ORACLE CORPORATION. *JDK 21 Releases*. Online. Oracle. 2025. Dostupné z: <https://jdk.java.net/21/>. [cit. 2025-02-25].
- [7] JETBRAINS. *IntelliJ IDEA*. Online. JetBrains. 2025. Dostupné z: <https://www.jetbrains.com/idea/>. [cit. 2025-02-25].
- [8] GLUON. *Scene Builder*. Online. Scene Builder. 2025. Dostupné z: <https://gluonhq.com/products/scene-builder/>. [cit. 2025-02-25].
- [9] THE APACHE SOFTWARE FOUNDATION. *Apache Maven*. Online. Apache Maven Project. 2025. Dostupné z: <https://maven.apache.org/>. [cit. 2025-02-25].
- [10] SCOTT CHACON. *Git*. Online. Git. 2025. Dostupné z: <https://git-scm.com/>. [cit. 2025-02-25].
- [11] ORACLE CORPORATION. *MySQL*. Online. MySQL. 2025. Dostupné z: <https://www.mysql.com/>. [cit. 2025-02-25].
- [12] GLUON. *JavaFX*. Online. JavaFX. 2025. Dostupné z: <https://openjfx.io/>. [cit. 2025-02-25].
- [13] GOOGLE INC. *Gson*. Online. Gson. 2025. Dostupné z: <https://github.com/google/gson>. [cit. 2025-02-25].
- [14] THE APACHE SOFTWARE FOUNDATION. *Dotenv*. Online. Dotsenv. 2025. Dostupné z: <https://github.com/cdimascio/dotenv-java>. [cit. 2025-02-25].
- [15] THE APACHE SOFTWARE FOUNDATION. *Apache POI*. Online. Apache POI. 2025. Dostupné z: <https://poi.apache.org/>. [cit. 2025-02-25].
- [16] WIKIPEDIA. *XML*. Online. XML. 2025. Dostupné z: <https://en.wikipedia.org/wiki/XML>. [cit. 2025-02-25].
- [17] WIKIPEDIA. *Parsing*. Online. Parsing. 2025. Dostupné z: <https://en.wikipedia.org/wiki/Parsing>. [cit. 2025-02-25].
- [18] WIKIPEDIA. *JSON*. Online. JSON. 2025. Dostupné z: <https://en.wikipedia.org/wiki/JSON>. [cit. 2025-02-25].
- [19] ORACLE CORPORATION. *MySQL Workbench*. Online. MySQL Workbench. 2025. Dostupné z: <https://www.mysql.com/products/workbench/>. [cit. 2025-02-25].

- [20] THE APACHE SOFTWARE FOUNDATION. *XAMPP*. Online. XAMPP. 2025. Dostupné z: <https://www.apachefriends.org/index.html>. [cit. 2025-02-25].
- [21] *Uživatelské rozhraní*. Online. Wikipedie. 2024. Dostupné z: https://cs.wikipedia.org/wiki/U%C5%BEivatelsk%C3%A9_rozhran%C3%AD. [cit. 2025-02-27].
- [22] *Entity–relationship model*. Online. Wikipedie. 2025. Dostupné z: https://en.wikipedia.org/wiki/Entity%E2%80%93relationship_model. [cit. 2025-02-27].
- [23] GOOGLE INC. *Algoritmus SHA-256*. Online. Algoritmus SHA-256. 2025. Dostupné z: <https://support.google.com/google-ads/answer/9004655?hl=cs>. [cit. 2025-02-27].
- [24] *JavaFX working with threads and GUI*. Online. StackOverflow. 2013. Dostupné z: <https://stackoverflow.com/questions/16708931/javafx-working-with-threads-and-gui>. [cit. 2025-03-01].
- [25] W3SCHOOLS. *Java Threads*. Online. W3Schools. 2025. Dostupné z: https://www.w3schools.com/java/java_threads.asp. [cit. 2025-03-01].
- [26] ORACLE CORPORATION. *Java Threads*. Online. Oracle® Database. 2025. Dostupné z: <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/index.html>. [cit. 2025-03-01].
- [27] MICROSOFT. *Getting started with ASP.NET MVC 5*. Online. Learn Microsoft. 2025. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/getting-started/introduction/getting-started>. [cit. 2025-03-01].
- [28] DIGITALOCEAN, LLC. *Logger in Java – Java Logging Example*. Online. DigitalOcean. 2025. Dostupné z: <https://www.digitalocean.com/community/tutorials/logger-in-java-logging-example>. [cit. 2025-03-01].
- [29] WIKIPEDIA. *Polymorfismus (programování)*. Online. Wikipedia. 2023. Dostupné z: [https://cs.wikipedia.org/wiki/Polymorfismus_\(programov%C3%A1n%C3%AD\)](https://cs.wikipedia.org/wiki/Polymorfismus_(programov%C3%A1n%C3%AD)). [cit. 2025-03-01].
- [30] OPENAI. *OpenAI GPT-3*. Online. Wikipedia. 2025. Dostupné z: <https://platform.openai.com/docs/models/gpt-3>. [cit. 2025-03-01].
- [31] OPENAI. *OpenAI developer platform*. Online. Wikipedia. 2025. Dostupné z: <https://platform.openai.com/docs/overview>. [cit. 2025-03-01].
- [32] IBM. *What is an API?* Online. Wikipedia. 2025. Dostupné z: <https://www.ibm.com/think/topics/api>. [cit. 2025-03-01].
- [33] *Framework search*. Online. TechTarget. 2025. Dostupné z: <https://www.techtarget.com/search>. [cit. 2025-03-01].
- [34] *Regular Expressions in Java*. Online. GeeksforGeeks. 2024. Dostupné z: <https://www.geeksforgeeks.org/regular-expressions-in-java/>. [cit. 2025-03-09].
- [35] *Java Lambda Expressions*. Online. GeeksforGeeks. 2025. Dostupné z: <https://www.geeksforgeeks.org/lambda-expressions-java-8/>. [cit. 2025-03-09].
- [36] *Java CSS*. Online. Pastebin. 2012. Dostupné z: <https://pastebin.com/0PebD9nR>. [cit. 2025-03-09].
- [37] *JavaFX CSS styling* [@BroCodez]. Online. 2021. Dostupné z: <https://www.youtube.com/watch?v=o-lAsVuskKI>. [cit. 2025-03-09].